

Data Analysis using Python

03 – Introduction to Plotting

Contents

Why data visualization is important

Anatomy of a figure

Univariate, bivariate and multivariate statistics

Matplotlib

Seaborn

Pandas.plot

Why Plotting is Important in Data Analysis

Visualizing Data Trends

Purpose: Identify patterns, trends, and relationships in data that are difficult to see in raw numerical form.

Example: A line plot showing stock prices over time highlights market trends more effectively than raw numbers.

Simplifying Complex Information

Purpose: Convert large datasets into easily digestible visual summaries.

Example: Histograms summarize distributions of data, such as customer age groups.

Supporting Decision-Making

Purpose: Data visualization enhances communication and aids in data-driven decision-making.

Example: A bar chart comparing revenue across regions helps stakeholders prioritize resources.

Identifying Anomalies

Purpose: Quickly detect outliers or anomalies that might skew results or indicate errors.

Example: Scatter plots reveal clusters or outliers in datasets.

Enhancing Communication

Purpose: Visuals make data accessible to a wider audience, including non-technical stakeholders.

Example: Pie charts highlight proportions for easy comparison.

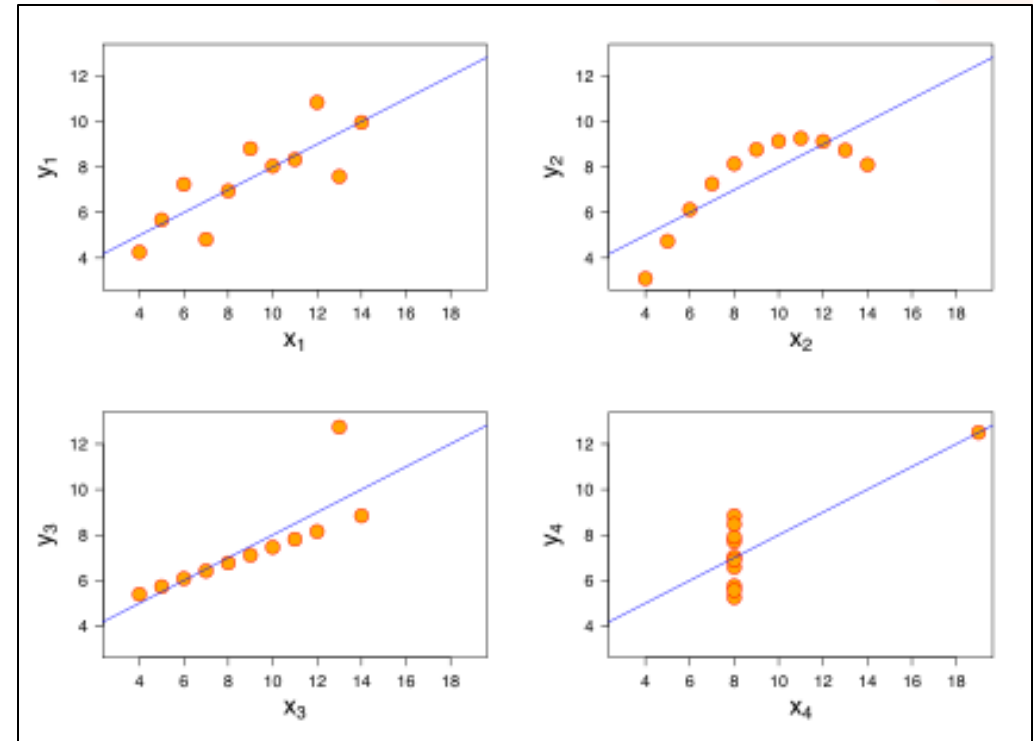
Anscombes Quartet

A good example of why visualizing is important is to plot Anscombes Quartet.

2 continuous X and Y variables

Each with same **mean**, **variance**, **correlation** and **regression**

When viewed, they produce 4 very different visuals



Anatomy of a figure

A **figure** is the top-level container for all plot elements.

Acts as the canvas where one or more plots (axes) are drawn.

Key Components:

Figure (Figure):

The entire window or page that contains the plot(s).
Created using `plt.figure()` or `plt.subplots()`.

Axes (Axes):

The individual plot or chart within the figure.
Contains the actual data representation, such as lines or bars.

Axis (Axis):

Represents the x-axis and y-axis in the plot.
Responsible for ticks, labels, and limits.

Title:

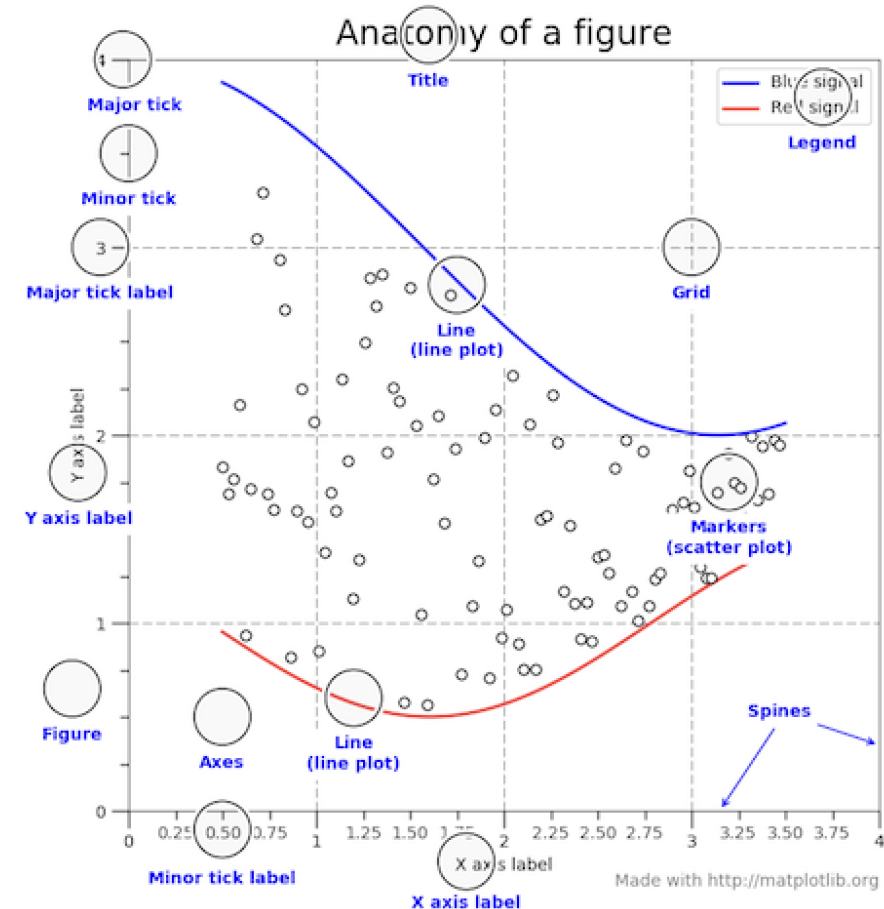
Provides a title for the entire figure or individual axes.

Legend:

Explains the elements of the plot (e.g., colors, markers).

Grid:

Optional gridlines for better readability of data points.



Univariate Statistics

Definition:

Analysis involving a **single variable**.

Focuses on summarizing and understanding the distribution, central tendency, and variability of the data.

Examples:

Histograms, box plots, and kernel density estimates (KDE).

Use Case:

Understanding the spread of ages in a population.

Bivariate Statistics

Definition:

Analysis involving **two variables**.

Examines relationships, associations, and correlations between the variables.

Examples:

Scatter plots, line plots, and heatmaps.

Use Case:

Analyzing the relationship between study hours and test scores.

Multivariate Statistics

Definition:

Analysis involving **three or more variables**.

Explores complex relationships and interactions between variables.

Examples:

Pair plots, 3D scatter plots, and clustered heatmaps.

Use Case:

Studying the interaction between age, income, and education level.

Matplotlib

matplotlib

What is Matplotlib?

A comprehensive library for creating static, animated, and interactive visualizations in Python. Designed for flexibility and compatibility with NumPy and other Python libraries.

Key Features:

Wide Range of Plots:

Line plots, bar plots, scatter plots, histograms, box plots, pie charts, and more.

Highly Customizable:

Control every element of a plot: labels, titles, gridlines, colors, legends, and more.

Integration with Other Libraries:

Works seamlessly with Pandas, NumPy, and Seaborn for advanced data visualizations.

Multiple Plot Management:

Supports multiple figures and axes for creating complex layouts.

Supports Various Output Formats:

Save plots in formats like PNG, JPG, SVG, PDF, and more.

matplotlib.pyplot.plot()

The **plot()** function in Matplotlib is used to create **2D line plots**.

One of the most fundamental functions for visualizing relationships between variables.

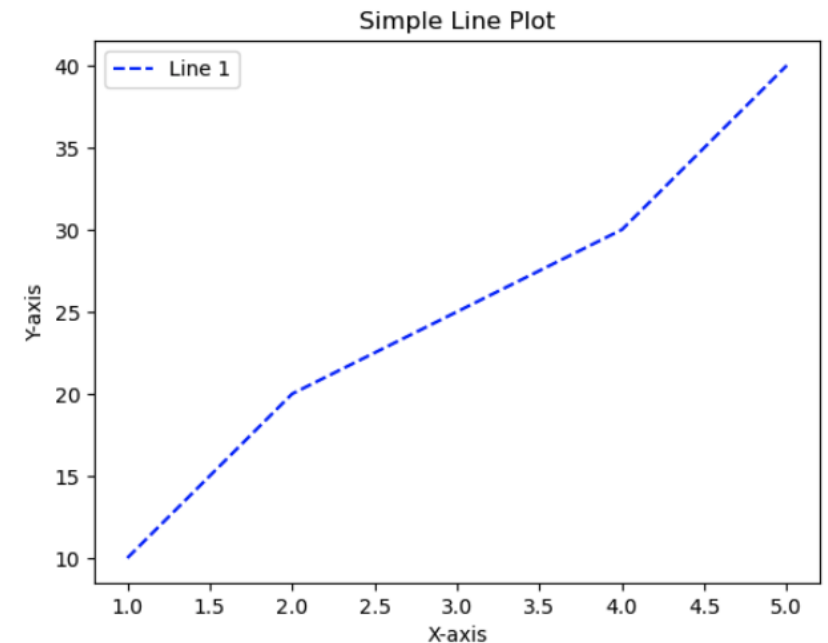
```
import matplotlib.pyplot as plt

# Sample data
x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]

# Create a line plot
plt.plot(x, y, label='Line 1', color='blue', linestyle='--')

# Add labels and title
plt.title('Simple Line Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()

# Display the plot
plt.show()
```



matplotlib.hist()

The **hist()** function in Matplotlib is used to create **histograms**.

A histogram is a graphical representation of the distribution of numerical data.

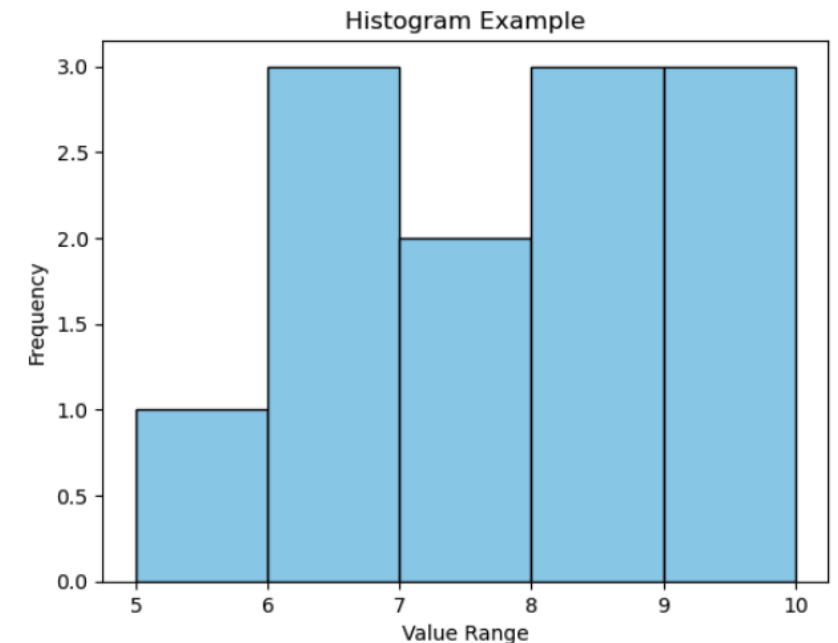
```
import matplotlib.pyplot as plt

# Sample data
data = [7, 8, 5, 6, 9, 8, 8, 10, 6, 7, 9, 6]

# Create a histogram
plt.hist(data, bins=5, color='skyblue', edgecolor='black')

# Add labels and title
plt.title('Histogram Example')
plt.xlabel('Value Range')
plt.ylabel('Frequency')

# Display the plot
plt.show()
```



matplotlib.scatter()

The **scatter()** function in Matplotlib is used to create **scatter plots**.

A scatter plot displays points representing the values of two variables, highlighting their relationship.

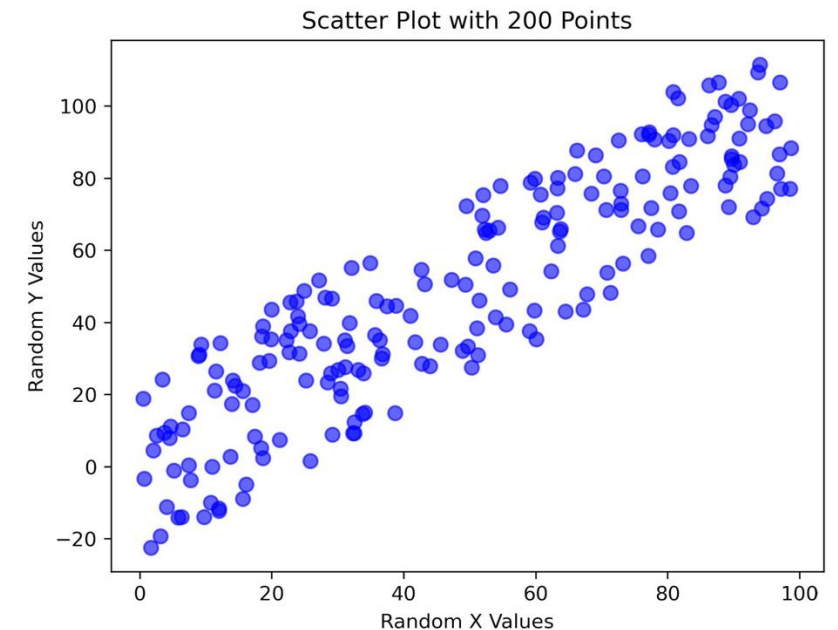
```
import matplotlib.pyplot as plt
import numpy as np

# Generate sample data
np.random.seed(42) # For reproducibility
x = np.random.rand(200) * 100 # 200 random values for X-axis
y = x + (np.random.rand(200) * 50 - 25) # Y-axis with some noise

# Create a scatter plot
plt.scatter(x, y, color='blue', marker='o', s=50, alpha=0.6)

# Add labels and title
plt.title('Scatter Plot with 200 Points')
plt.xlabel('Random X Values')
plt.ylabel('Random Y Values')

# Display the plot
plt.show()
```



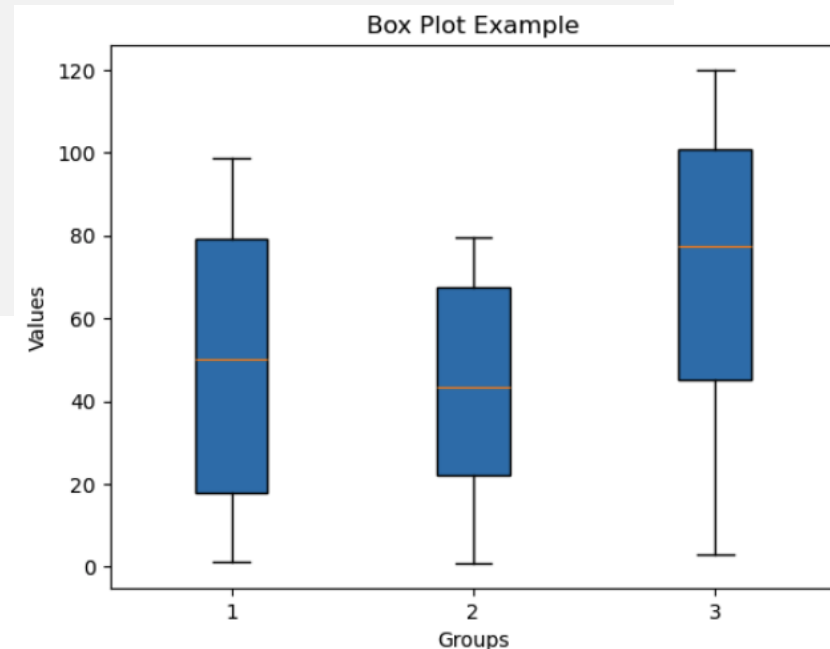
matplotlib.boxplot()

The **boxplot()** function in Matplotlib creates **box-and-whisker plots**.
Used to visualize the **distribution**, **variability**, and **outliers** in data.

```
import matplotlib.pyplot as plt
import numpy as np

# Sample data
data = [np.random.rand(50) * 100, np.random.rand(50) * 80, np.random.rand(50) * 120]

# Create a box plot
plt.boxplot(data, patch_artist=True)
plt.title('Box Plot Example')
plt.xlabel('Groups')
plt.ylabel('Values')
plt.show()
```



matplotlib.scatter()

A **Bi variate** scatter

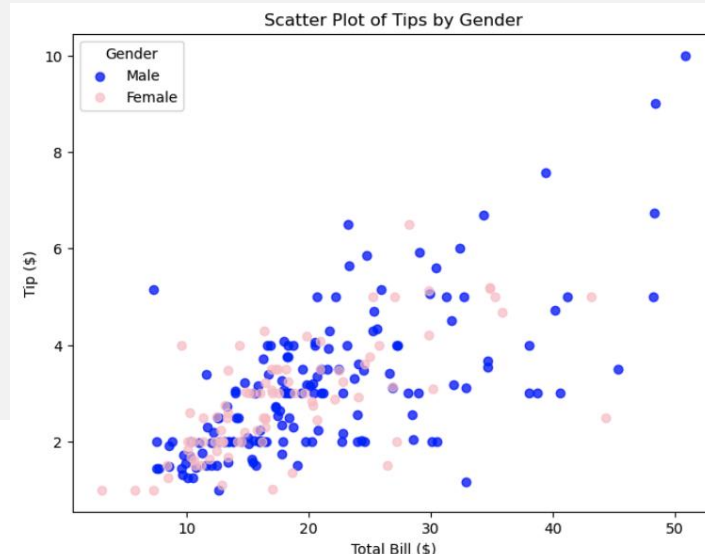
```
import matplotlib.pyplot as plt
import seaborn as sns

# Load the tips dataset from Seaborn
tips = sns.load_dataset('tips')

# Create a scatter plot
plt.figure(figsize=(8, 6))
for gender, color in zip(['Male', 'Female'], ['blue', 'pink']):
    subset = tips[tips['sex'] == gender]
    plt.scatter(subset['total_bill'], subset['tip'], label=gender, alpha=0.7, c=color)

# Add labels and title
plt.title('Scatter Plot of Tips by Gender')
plt.xlabel('Total Bill ($)')
plt.ylabel('Tip ($)')
plt.legend(title='Gender')

# Display the plot
plt.show()
```



seaborn

seaborn

What is Seaborn?

A Python library built on top of Matplotlib, designed for statistical data visualization. Provides high-level functions for creating visually appealing and informative plots.

Key Features:

Simplified Syntax:

Create complex visualizations with just a few lines of code.

Example: **`sns.scatterplot(x='column1', y='column2', data=df)`**

Integrated with Pandas:

Works seamlessly with Pandas DataFrames for quick and easy data exploration.

Statistical Insights:

Built-in functions for visualizing distributions, relationships, and categorical data.

Predefined Aesthetic Styles:

Offers beautiful themes and palettes out of the box (e.g., **darkgrid**, **whitegrid**).

Advanced Plots:

Includes high-level functions for **pair plots**, **violin plots**, **heatmaps**, and **regression plots**.

seaborn.histplot()

Seaborn's **histplot()** function simplifies creating informative **histograms**.

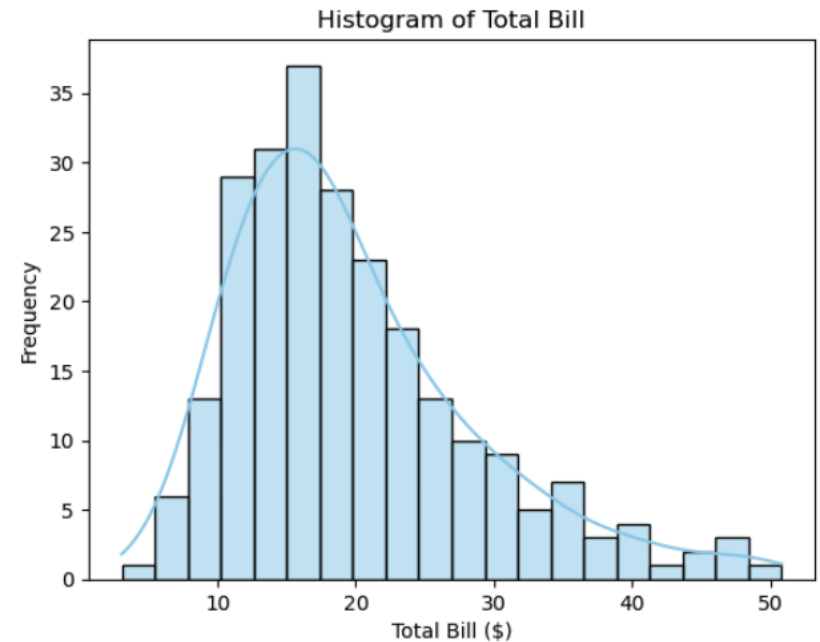
```
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
data = sns.load_dataset('tips')

# Create a histogram
sns.histplot(data['total_bill'], bins=20, kde=True, color='skyblue')

# Add labels and title
plt.title('Histogram of Total Bill')
plt.xlabel('Total Bill ($)')
plt.ylabel('Frequency')

# Show the plot
plt.show()
```



seaborn.displot()

A Seaborn function for **distribution plots** that combines histogram and kernel density estimation (KDE) with modern enhancements.

Creates a **FacetGrid**, allowing for easy faceting by categorical variables.

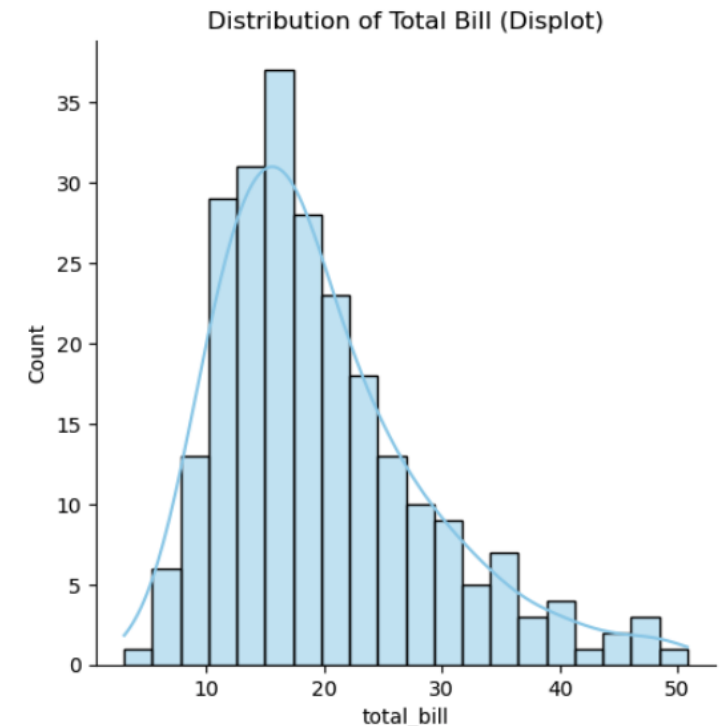
```
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
data = sns.load_dataset('tips')

# Create a displot
sns.displot(data=data, x='total_bill', bins=20, kde=True, color='skyblue')

# Add title
plt.title('Distribution of Total Bill (Displot)', loc='center')

# Show the plot
plt.show()
```



seaborn.kdeplot()

A Seaborn function that generates a **Kernel Density Estimate (KDE)** plot. Visualizes the **probability density** of a dataset as a smooth curve.

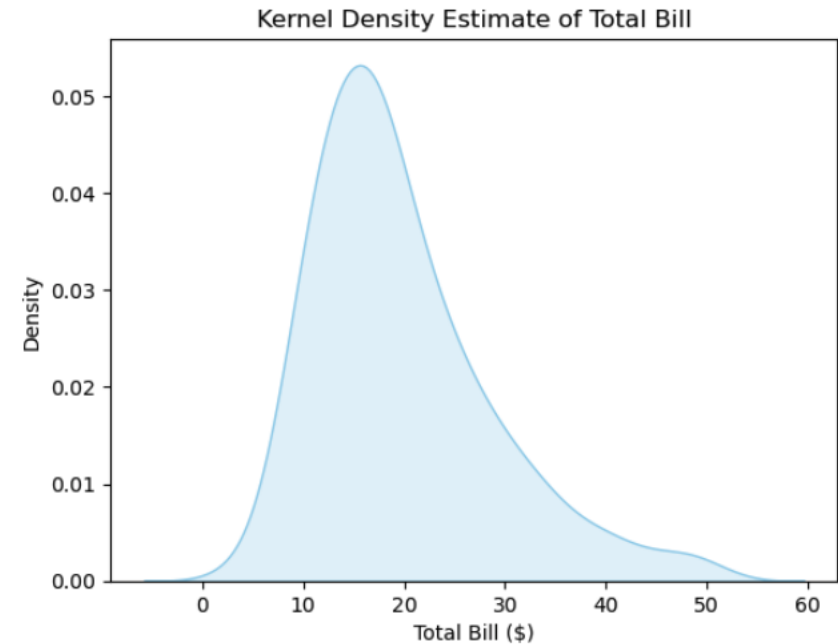
```
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
data = sns.load_dataset('tips')

# Create a KDE plot
sns.kdeplot(data=data['total_bill'], fill=True, color='skyblue')

# Add labels and title
plt.title('Kernel Density Estimate of Total Bill')
plt.xlabel('Total Bill ($)')
plt.ylabel('Density')

# Show the plot
plt.show()
```



seaborn.rugplot()

A Seaborn function that adds a **rug plot**—a series of small vertical ticks at the base of a plot.

Provides a compact visual representation of the **distribution of individual data points**.

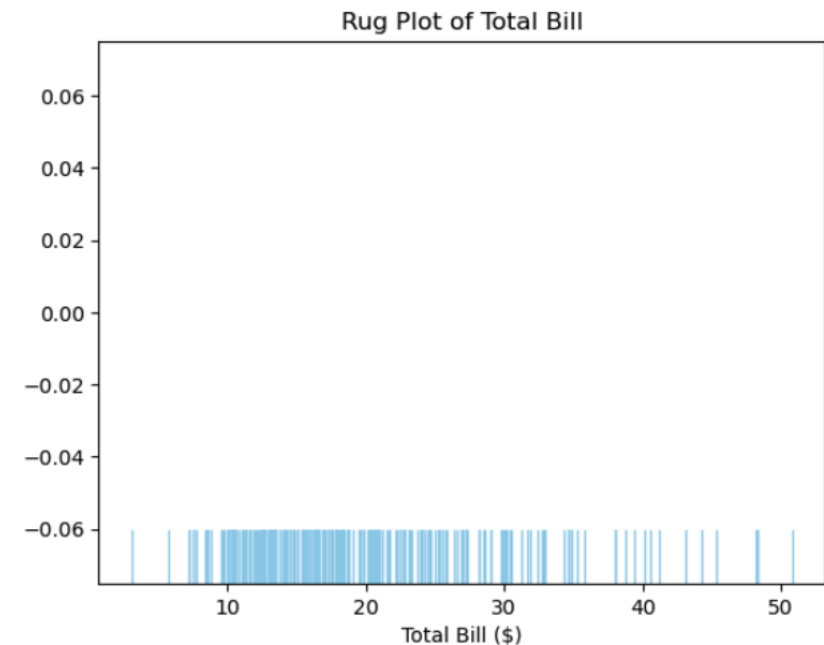
```
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
data = sns.load_dataset('tips')

# Create a rug plot
sns.rugplot(data=data['total_bill'], height=0.1, color='skyblue')

# Add labels and title
plt.title('Rug Plot of Total Bill')
plt.xlabel('Total Bill ($)')

# Show the plot
plt.show()
```



seaborn.barplot()

A Seaborn function that creates a **bar plot** to display aggregate values for categorical data.

Visualizes **central tendencies** like **mean** or **median** with error bars.

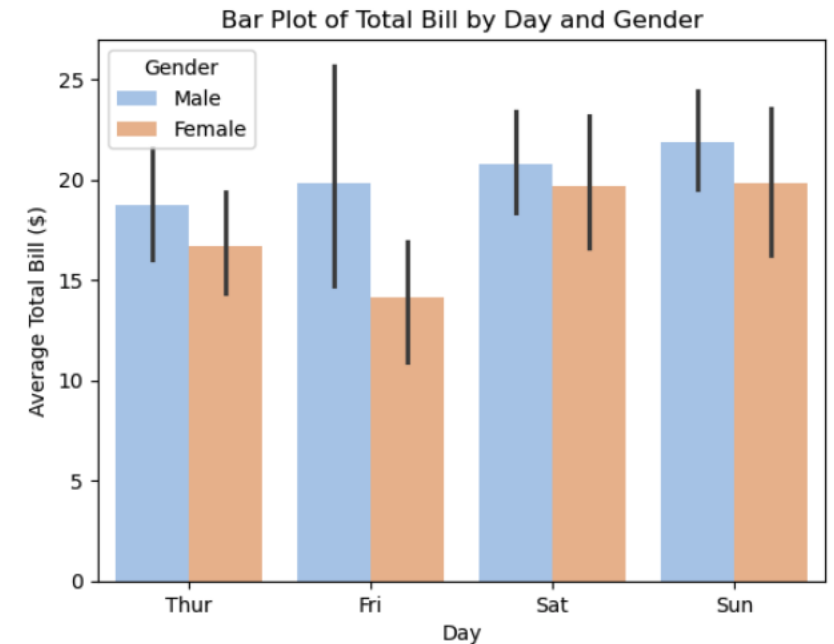
```
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
data = sns.load_dataset('tips')

# Create a bar plot
sns.barplot(data=data, x='day', y='total_bill', hue='sex', palette='pastel')

# Add labels and title
plt.title('Bar Plot of Total Bill by Day and Gender')
plt.xlabel('Day')
plt.ylabel('Average Total Bill ($)')
plt.legend(title='Gender')

# Show the plot
plt.show()
```



seaborn.scatterplot()

A Seaborn function that creates a **bar plot** to display aggregate values for categorical data.

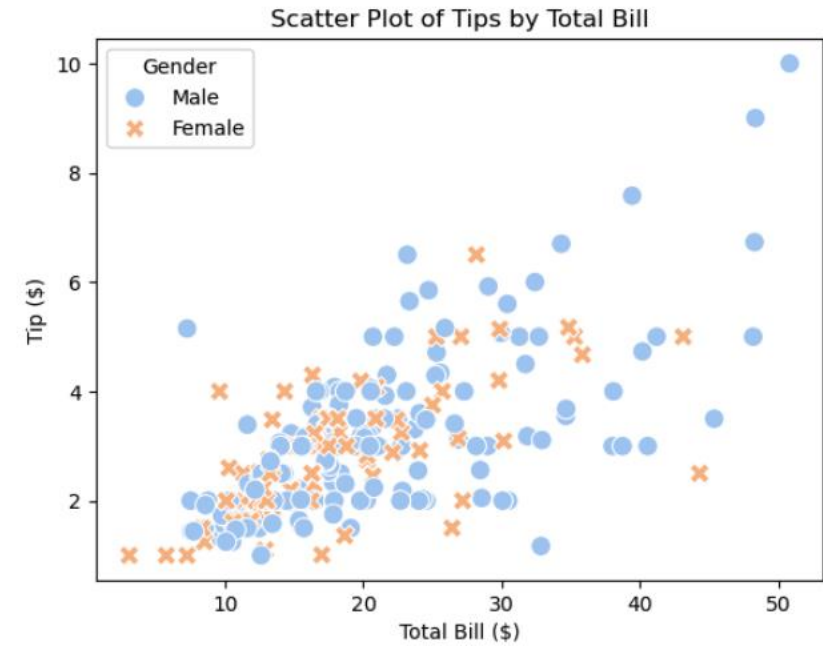
```
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
data = sns.load_dataset('tips')

# Create a scatter plot
sns.scatterplot(data=data, x='total_bill', y='tip', hue='sex',
               style='sex', palette='pastel', s=100)

# Add labels and title
plt.title('Scatter Plot of Tips by Total Bill')
plt.xlabel('Total Bill ($)')
plt.ylabel('Tip ($)')
plt.legend(title='Gender')

# Show the plot
plt.show()
```



seaborn.violinplot()

What is violinplot?

A Seaborn function that combines features of **box plots** and **kernel density plots**. Visualizes the distribution of a dataset and its probability density.

Key Features of violinplot:

Distribution and Summary Statistics:

Displays the full data distribution along with key summary statistics (median, quartiles).

Split by Categories:

Compare distributions across different groups using hue.

Highly Customizable:

Adjust bandwidth, split the violin, or scale the plot by sample size.

seaborn.violinplot()

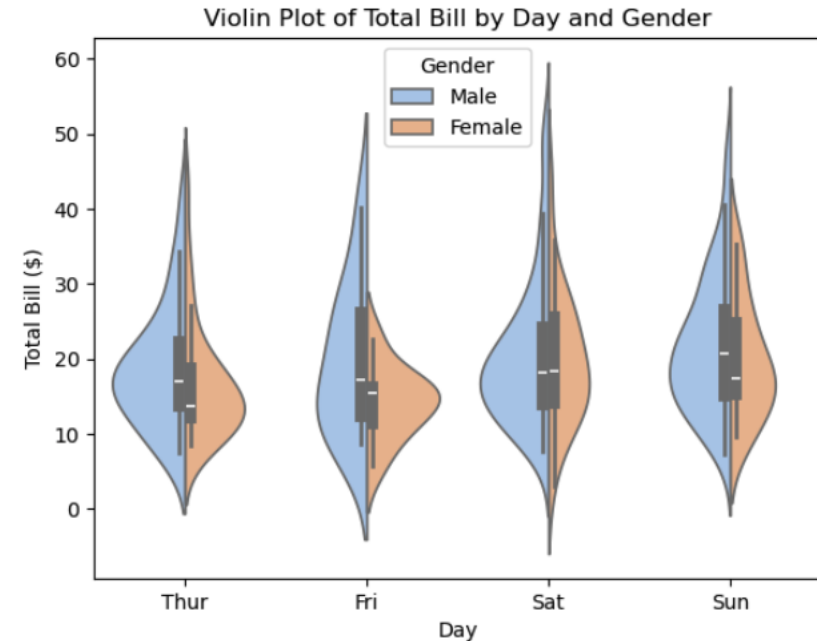
```
import seaborn as sns
import matplotlib.pyplot as plt

# Sample data
data = sns.load_dataset('tips')

# Create a violin plot
sns.violinplot(data=data, x='day', y='total_bill', hue='sex',
               split=True, palette='pastel')

# Add labels and title
plt.title('Violin Plot of Total Bill by Day and Gender')
plt.xlabel('Day')
plt.ylabel('Total Bill ($)')
plt.legend(title='Gender')

# Show the plot
plt.show()
```



Features Highlighted in the Example:

Split Violin

split=True shows male and female distributions side-by-side for each day.

Custom Palette

palette='pastel' provides visually appealing colours.

Bandwidth Control

Automatically adjusts for smoother density estimation.

pandas.plot

pandas.plot

What is the Pandas plot Module?

A high-level interface for generating **quick and simple visualizations** directly from Pandas DataFrames or Series.

Built on top of **Matplotlib**, but with a simpler syntax for data exploration.

pandas.plot – key features

Ease of Use:

Automatically infers plot types and axes from the data structure.
Minimal code to generate visualizations.

Supports Multiple Plot Types:

Common options include:

Line plot: `.plot(kind='line')` (default).

Bar plot: `.plot(kind='bar' or 'barh')`.

Histogram: `.plot(kind='hist')`.

Scatter plot: `.plot(kind='scatter')`.

Box plot: `.plot(kind='box')`.

Area plot: `.plot(kind='area')`.

Pie chart: `.plot(kind='pie')`.

pandas.plot – key features

Integration with DataFrames:

Works seamlessly with DataFrame columns and indices for automatic labeling.

Customization:

Allows control over plot elements like titles, labels, colors, and legends.

Built-In Grouping:

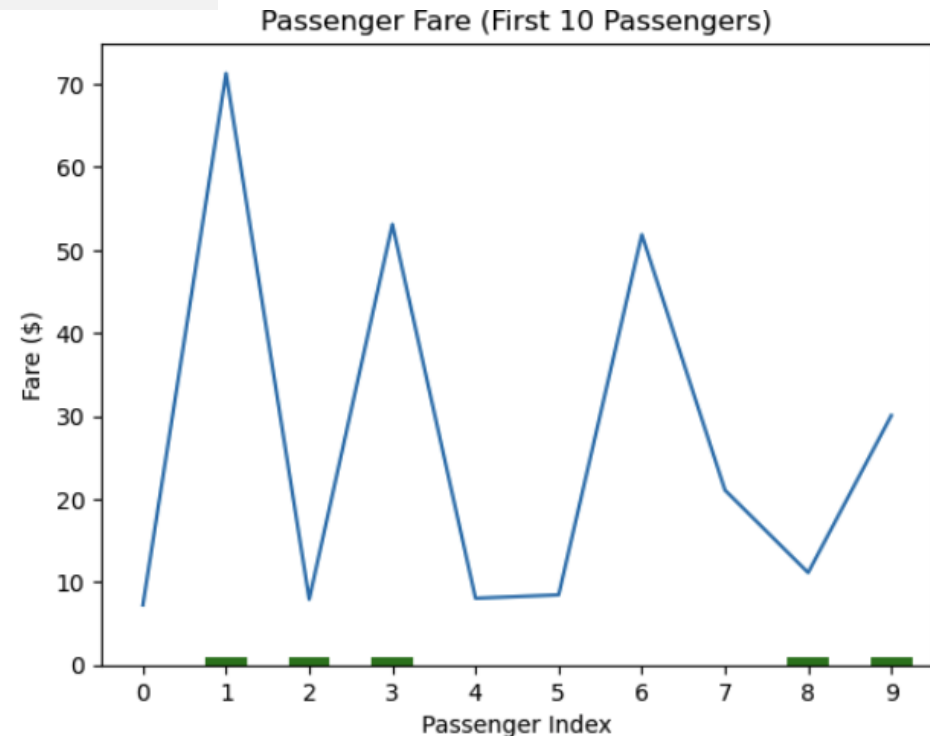
Supports grouped data visualization using the `by` parameter.

pandas.plot – line plot

```
# Import necessary libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Titanic dataset from Seaborn
titanic = sns.load_dataset('titanic')

titanic['fare'][:10].plot(kind='line',
                          title='Passenger Fare (First 10 Passengers)')
plt.xlabel('Passenger Index')
plt.ylabel('Fare ($)')
plt.show()
```



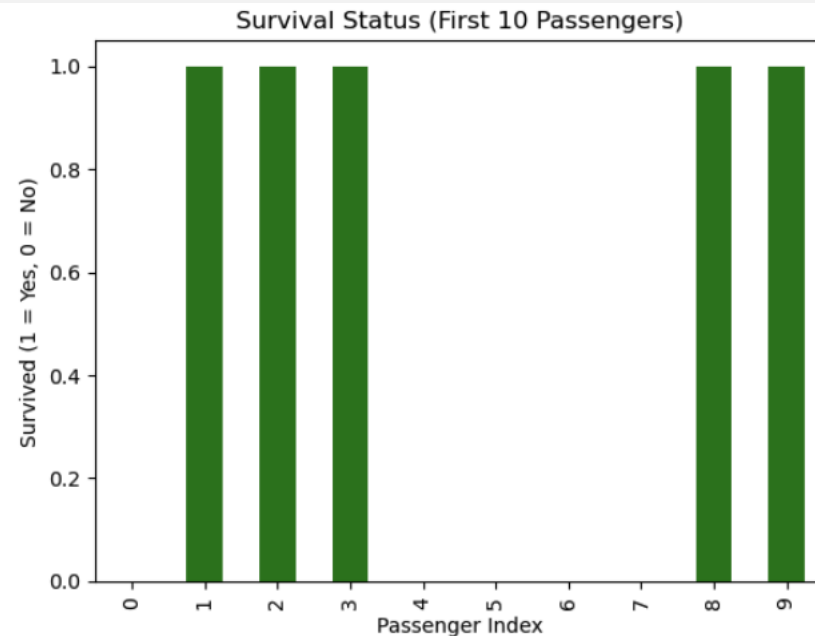
pandas.plot – bar plot

```
# Import necessary libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Titanic dataset from Seaborn
titanic = sns.load_dataset('titanic')

titanic['survived'][:10].plot(kind='bar',
                             title='Survival Status (First 10 Passengers)',
                             color=['green' if s else 'red' for s in titanic['survived'][:10]])

plt.xlabel('Passenger Index')
plt.ylabel('Survived (1 = Yes, 0 = No)')
plt.show()
```



pandas.plot – hist plot

```
# Import necessary libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Titanic dataset from Seaborn
titanic = sns.load_dataset('titanic')

titanic['age'].plot(kind='hist', bins=20,
                    title='Distribution of Passenger Ages',
                    color='skyblue', edgecolor='black')

plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```

